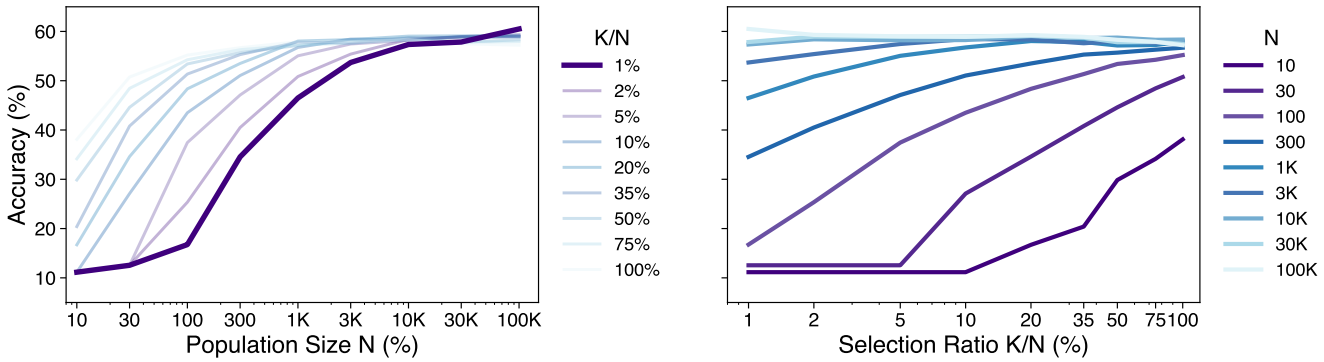

Figure 10: Scaling curves for population size K and ensemble size N on Countdown task.

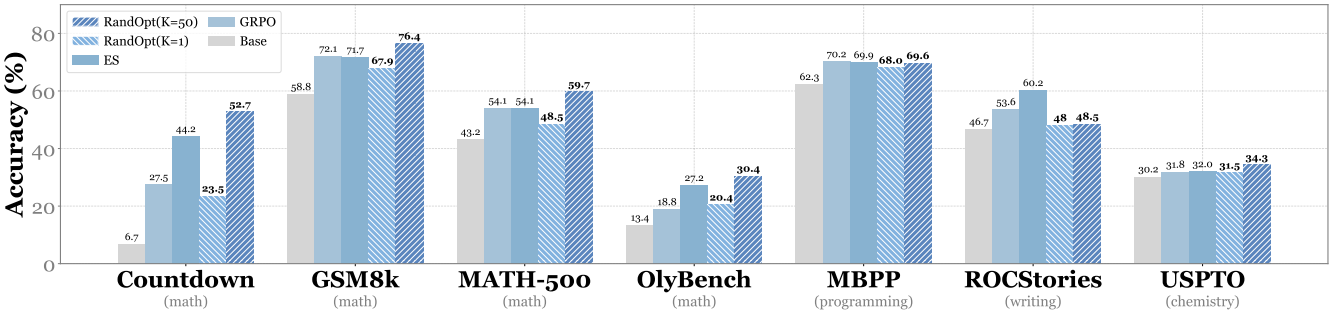

Figure 11: Model performance comparison on Countdown, GSM8k, MATH-500, and OlympiadBench using the Qwen2.5-1.5B-Instruct model. More results for additional models and baselines can be found in Appendix Table 4.

C Baseline Methods

Test-Time Majority Vote. Majority voting (Wang et al., 2023) is a test-time inference strategy that samples multiple independent responses from a model and selects the most frequently occurring answer. This approach improves accuracy without updating model parameters by leveraging the diversity of sampled reasoning paths.

Best-of-N. Best-of-N (Ouyang et al., 2022) samples multiple responses at inference time and selects the highest-scoring one according to a predefined evaluation metric or reward function, rather than aggregating answers by frequency as in majority voting.

PPO. Proximal Policy Optimization (Schulman et al., 2017) uses a clipped surrogate objective to stabilize policy updates in RLHF, requiring both a policy model and a critic network.

GRPO. Group Relative Policy Optimization (Shao et al., 2024) removes the critic by computing advantages from group-level reward statistics, reducing memory overhead compared to PPO.

ES. Evolution Strategies at Scale (Qiu et al., 2025) perform gradient-free optimization by perturbing parameters with Gaussian noise and updating based on fitness-weighted perturbations.

D Additional Experimental Analysis

Full results on LLM For post-training LLMs, Table 4 reports our full results across different base models, adaptation methods, and tasks.

Effect of ensembling Figure 11 compares RandOpt (K=1) to RandOpt (K=50), alongside several baselines. These results show that ensembling over many perturbations is critical to getting competitive performance on most tasks, but also that even without ensembling (just taking the top perturbation; K=1), we observe a substantial

performance boost over the base model.

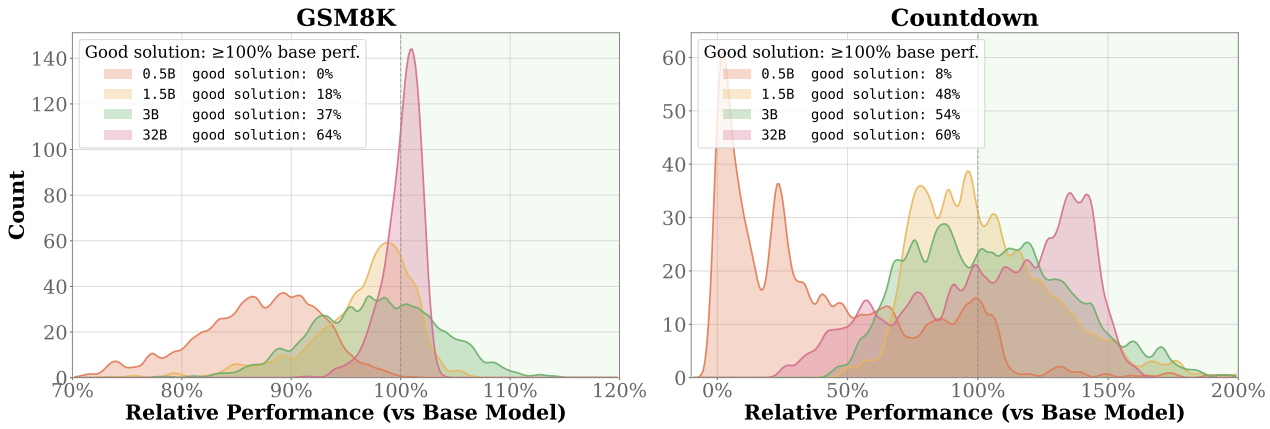

Figure 12: Performance distributions of 500 randomly perturbed models relative to base model accuracy on the math reasoning task GSM8k (left) and Countdown (right). The fraction of perturbations matching or exceeding base performance (shaded region) increases with model size: 0% → 64% on GSM8K and 8% → 60% on Countdown as model size grows from 0.5B to 32B parameters. Note: x-axis shows relative performance; longer tails in smaller models do not equate to higher absolute accuracy.

Solution density histograms In Figure 12, we show the full distribution of performance improvement over random weight perturbations for GSM8K and Countdown, on the Qwen2.5-Instruct series of models.

Log-linear correlation between population size (N) and performance. Appendix Figure 10 illustrates the scaling properties of our method on the Countdown task. As shown in the left panel, we observe a log-linear correlation between population size and performance across different selection ratios. RandOpt benefits significantly from scaling N in the population size.

Selection ratio saturation. The right panel shows that while increasing the selection ratio significantly improves performance when N is small, this benefit saturates as N scales up. For sufficiently large N, the minimal value of K (selecting only the top 1%) yields high performance. This demonstrates that a large population size N allows for a very small topK selection, which can reduce inference time costs.

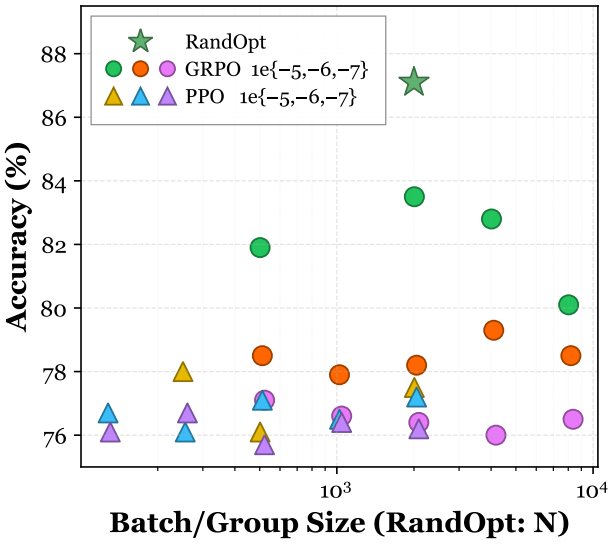

Figure 13: Batch/group size vs. accuracy under one-step training. Scaling baseline parallelism does not get RandOpt's performance. Task: GSM8K, Model: Qwen2.5-3B-Instruct, N=5000, K=50)

Scaling Parallelism of the Baselines Does Not Match RandOpt (1-Step Training). Figure 13 plots batch/-group size versus accuracy on GSM8K using Qwen2.5-3B-Instruct, with all methods run for one training step. The key takeaway is that using larger batch/group size does not really help. We grid-search GRPO and PPO on 8 GPUs over learning rates $1e-5$, $1e-6$, and $1e-7$; batch size (PPO) ranges from 128 to 2048, and group size (GRPO) ranges from 512 to 8192. The x-axis is batch/group size (RandOpt uses population size N), color encodes learning rate, and marker shape denotes algorithm.

PPO does not benefit consistently from larger batches: best accuracy is 78.0% at batch size 256 (lr=$1e-5$), while larger-batch runs such as batch size 2048 reach at most 77.5%.

GRPO reaches its peak at 83.5% (group size=512 × 4, lr=$1e-5$), but increasing group size does not improve performance (e.g., 80.1% at 2048 × 4 with lr=$1e-5$).

RandOpt ($N = 3000$) reaches 87.1%, exceeding all GRPO and PPO settings. This indicates that scaling baseline parallelism does not close the gap with RandOpt 's performance.

E Implementation Details

All experiments are conducted on NVIDIA GH200 GPUs. To ensure a fair comparison, we normalize the computational budgets of all methods by the total training FLOPs. For the experiments in Table 4, We have two variants that differ in their selection strategy: RandOpt (random) performs selection by random sampling, while RandOpt (ES) updates the population using an evolution strategies (ES) algorithm and selects the top-50 models from the final ES population. We use a population size of 5,000 and ensemble the top-50 models for RandOpt (random), and a population size of 100 with 50 iterations and ensemble the top-50 models for RandOpt (ES). For the figures in Section 2, we use $\sigma = 0.005$. For all datasets, we use the first 200 samples from the training set as the RandOpt training set, and all samples from the test set for evaluation. For MATH-500, we manually split the dataset: the first 200 samples are used for training and the remaining samples for testing. For OlympiadBench, we use the text-only English competition math subset (`OE_TO_maths_en_COMP.parquet`), since most other subsets require visual inputs.

E.1 FLOPs Calculation

For a model with P parameters and a sequence length of L, we follow the standard estimation where a single forward pass requires approximately $2PL$ FLOPs, and a backward pass requires $4PL$ FLOPs (Narayanan et al., 2021). The total training compute is determined by the number of samples processed and the computational overhead per sample.

E.2 Method-Specific Compute

- **GRPO**: Each step processes a batch of B questions with G responses per question. Each response involves a policy forward, a reference model forward, and a policy backward pass.

$$\text{FLOPs}_{\text{GRPO}} = T_{\text{GRPO}} \cdot B \cdot G \cdot \underbrace{(2+2+4)}_{\text{fwd + ref + bwd}} \cdot PL = \mathbf{8} \cdot \mathbf{T}_{\text{GRPO}} \cdot \mathbf{B} \cdot \mathbf{G} \cdot \mathbf{PL} \tag{7}$$

- **PPO**: Adds critic forward and backward passes to the GRPO baseline.

$$\text{FLOPs}_{\text{PPO}} = T_{\text{PPO}} \cdot B \cdot G \cdot \underbrace{(2+2+2+4+4)}_{\text{fwd + ref + crit_fwd + bwd + crit_bwd}} \cdot PL = \mathbf{14} \cdot \mathbf{T}_{\text{PPO}} \cdot \mathbf{B} \cdot \mathbf{G} \cdot \mathbf{PL} \tag{8}$$

- **ES & RandOpt**: These gradient-free methods only require forward passes for evaluation. For a population size N and evaluation dataset size D:

$$\text{FLOPs}_{\text{ES/RandOpt}} = T_{\text{ES}} \cdot N \cdot D \cdot \underbrace{2}_{\text{fwd}} \cdot PL = \mathbf{2} \cdot \mathbf{T}_{\text{ES}} \cdot \mathbf{N} \cdot \mathbf{D} \cdot \mathbf{PL} \tag{9}$$

E.3 Hyperparameters

Appendix Table 3 shows the hyperparemeters across methods. To ensure a fair comparison, we align the hyperparameters such that all methods consume equivalent total training FLOPs. We balance the batch size and iteration counts to account for algorithmic overheads. For instance, GRPO uses a larger batch size ($B = 1024$) compared to PPO ($B = 128$) due to the latter's additional memory cost for the critic network. Similarly, we match the total number of sample evaluations between the iterative ES (30 population $\times$ 167 steps) and the single-step RandOpt (5000 population $\times$ 1 step). All experiments utilize bfloat16 precision with a maximum sequence length of 1024.

F Additional Results on 1D Signals

We show additional results of the experiments on 1D signals in Tables 5 and 6. Table 5 tests the approximation ability of RandOpt: here we select perturbations based on their ability to fit a single test function (plotted in blue).